TRANSITAI

AI-Based Smart Public Transport System

Crowd Prediction & Route Optimization Using Artificial Intelligence

PROTOTYPE – FULL CODING & TECHNICAL DOCUMENTATION

AI Immersion Project | Computer Science and Engineering

DETECT → PREDICT → OPTIMIZE → ACT

1. Abstract

TransitAI is an academic proof-of-concept for intelligent public transportation. It predicts near-future passenger demand, identifies possible overcrowding, displays bus status and supports better bus allocation. The prototype uses simulated transport data so that the demonstration is reproducible, while the code is structured to allow later connection to real GPS, traffic and passenger-count data.

2. Problem Statement

Public transport demand changes by time, day, route, traffic, weather and special events. Fixed schedules may therefore produce overcrowded buses on high-demand routes while other buses operate below capacity.

Problem statement: How can Artificial Intelligence predict passenger demand and optimize public transport routes before overcrowding occurs?

3. Objectives

• Predict passenger demand for a selected route and time.

• Estimate occupancy and classify overcrowding risk.

• Recommend additional bus deployment or reallocation.

• Display bus locations, ETA, occupancy and alerts.

• Recommend less-crowded alternatives to passengers.

• Keep AI and optimization services modular for future ML integration.

4. Existing vs Proposed System

Conventional	TransitAI Prototype
Fixed schedules	Demand-aware recommendations
Reactive crowd handling	Predictive overcrowding warning
Manual allocation	AI-assisted allocation
Basic bus tracking	Tracking + occupancy + ETA
Limited analytics	Demand, route and fleet analytics

5. Proposed Modules

• Landing/Home dashboard.

• Live bus tracking with map and bus details.

• AI crowd prediction with current vs predicted occupancy.

• Route optimization and AI recommendations.

• Admin dashboard with approval workflow.

• Passenger view with bus recommendations.

• Analytics and alerts.

6. Functional Requirements

• Navigation between modules.

• Prediction accepts route/time and returns occupancy, risk and recommendation.

• Optimization ranks routes using demand, capacity, traffic and waiting time.

• Admin can approve a recommendation and update bus state.

• Live demo mode changes bounded bus and occupancy values.

• Fallback mock data keeps the prototype usable without external services.

7. System Architecture

8. Data Flow

9. Technology Stack

• Frontend: React + Vite, CSS, Recharts, Leaflet.

• Backend: Node.js + Express.

• Prototype AI: transparent weighted prediction service; replaceable by Python/Scikit-learn later.

• Database: JSON mock data for prototype; MongoDB/Firebase for production.

• Maps: Leaflet + OpenStreetMap.

• Version control: Git.

10. Project Folder Structure

```
transitai/
■■■ client/
■   ■■■■ src/
■   ■   ■■■■ components/
■   ■   ■■■■ pages/
■   ■   ■■■■ services/
■   ■   ■   ■■■■ predictionService.js
■   ■   ■   ■■■■ optimizationService.js
■   ■   ■■■■ data/mockData.js
■   ■   ■■■■ App.jsx
■   ■   ■■■■ main.jsx
■   ■■■■ package.json
■■■■ server/
■   ■■■■ data/
■   ■■■■ services/
■   ■■■■ server.js
■■■■ README.md
```

11. Prototype Dataset

```
const buses = [
  { id:"B24", route:"R12", capacity:100, occupancy:82, status:"On Time" },
  { id:"B17", route:"R05", capacity:100, occupancy:32, status:"On Time" },
  { id:"B08", route:"R07", capacity:100, occupancy:76, status:"Delayed" }
];

const historicalDemand = [
  { route:"R12", hour:8, day:1, passengers:96 },
  { route:"R12", hour:9, day:1, passengers:71 },
  { route:"R07", hour:8, day:1, passengers:48 },
  { route:"R05", hour:8, day:1, passengers:32 }
];
```

12. Prediction Algorithm

For the academic prototype, the prediction service uses a transparent weighted model. It demonstrates the complete decision flow without falsely claiming a trained production ML model. The service boundary can later call a real trained model.

```
function predictOccupancy({current, historical, peak, traffic, event}) {
  const raw =
    current * 0.55 +
    historical * 0.25 +
    peak * 0.10 +
    traffic * 0.05 +
    event * 0.05;

  return Math.min(100, Math.max(0, Math.round(raw)));
}

function riskLevel(occupancy) {
  if (occupancy >= 90) return "HIGH";
  if (occupancy >= 75) return "MEDIUM";
  return "LOW";
}
```

13. Backend – Express API

```
const express = require("express");
const cors = require("cors");
const { predictForRoute } = require("./services/predictionService");
const { optimizeRoutes } = require("./services/optimizationService");

const app = express();
app.use(cors());
app.use(express.json());

app.get("/api/buses", (req,res) => res.json(buses));
app.get("/api/routes", (req,res) => res.json(routes));

app.post("/api/predict", (req,res) => {
  res.json(predictForRoute(req.body.routeId, req.body.time));
});

app.post("/api/optimize", (req,res) => {
  res.json(optimizeRoutes(req.body.routes, req.body.availableBuses));
});

app.listen(5000, () =>
  console.log("TransitAI API running on port 5000")
);
```

14. Prediction Service

```
function predictForRoute(routeId, time) {
  const p = {
    R12:{current:82,historical:95,peak:98,traffic:90,event:60},
    R07:{current:76,historical:72,peak:78,traffic:65,event:40},
    R05:{current:32,historical:38,peak:45,traffic:30,event:20}
  }[routeId] || {current:50,historical:50,peak:50,traffic:50,event:20};

  const predicted = Math.min(100, Math.round(
    p.current*0.55 + p.historical*0.25 +
    p.peak*0.10 + p.traffic*0.05 + p.event*0.05
  ));

  return {
    routeId, time,
    currentOccupancy:p.current,
    predictedOccupancy:predicted,
    risk: predicted>=90 ? "HIGH" :
          predicted>=75 ? "MEDIUM" : "LOW",
    windowMinutes:15
  };
}

module.exports = { predictForRoute };
```

15. Route Optimization Service

```
function optimizeRoutes(routes, availableBuses) {
  return routes.map(route => {
    const capacityRisk = route.demand / route.capacity;
    const trafficPenalty =
      route.traffic === "High" ? 0.15 :
      route.traffic === "Medium" ? 0.08 : 0.02;

    const score =
      capacityRisk*0.60 +
      route.waitingTime*0.20 +
      trafficPenalty*0.20;

    return {
      routeId: route.routeId,
      score: Number(score.toFixed(3)),
      recommendation:
        route.demand > route.capacity
          ? "Deploy additional bus"
          : route.demand < route.capacity*0.40
            ? "Consider reallocation"
            : "Keep current allocation"
    };
  }).sort((a,b) => b.score-a.score);
}
```

16. React Crowd Prediction Component

```
import { useState } from "react";
import { predictDemand } from "../services/predictionService";

export default function CrowdPrediction() {
  const [route,setRoute] = useState("R12");
  const [result,setResult] = useState(null);

  function handlePredict() {
    setResult(predictDemand(route,"08:15"));
  }

  return (
    <section className="prediction-page">
      <h1>AI Crowd Prediction</h1>
      <select value={route}
        onChange={e => setRoute(e.target.value)}>
        <option value="R12">Route 12</option>
        <option value="R07">Route 7</option>
        <option value="R05">Route 5</option>
      </select>
      <button onClick={handlePredict}>Predict Demand</button>

      {result && (
        <div className="prediction-card">
          <strong>{result.predicted}%</strong>
          <span>{result.risk} RISK</span>
          <p>{result.message}</p>
        </div>
      )}
    </section>
  );
}
```

17. Frontend Prediction Service

```
const profiles = {
  R12:{current:82,historical:95,peak:98,traffic:90,event:60},
  R07:{current:76,historical:72,peak:78,traffic:65,event:40},
  R05:{current:32,historical:38,peak:45,traffic:30,event:20}
};

export function predictDemand(routeId,time) {
  const p = profiles[routeId];
  const predicted = Math.min(100, Math.round(
    p.current*0.55 + p.historical*0.25 +
    p.peak*0.10 + p.traffic*0.05 + p.event*0.05
  ));
  const risk = predicted>=90 ? "HIGH" :
               predicted>=75 ? "MEDIUM" : "LOW";

  return {
    routeId,time,current:p.current,predicted,risk,
    message: risk==="HIGH"
      ? `Overcrowding likely in 15 minutes. Deploy an additional bus to ${routeId}.`
      : "Current allocation can be monitored."
  };
}
```

18. Live Map Component

```
import { MapContainer, TileLayer, Marker, Popup } from "react-leaflet";
import "leaflet/dist/leaflet.css";

export default function BusMap({ buses }) {
  return (
    <MapContainer center={[13.0827,80.2707]} zoom={12}
      style={{height:"520px",width:"100%"}}>
      <TileLayer
        attribution="&copy; OpenStreetMap contributors"
        url="https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png"
      />
      {buses.map(bus => (
        <Marker key={bus.id} position={bus.position}>
          <Popup>
            <b>{bus.id}</b><br/>
            {bus.route}<br/>
            Occupancy: {bus.occupancy}%<br/>
            ETA: {bus.eta} min
          </Popup>
        </Marker>
      ))}
    </MapContainer>
  );
}
```

19. Admin Recommendation Approval

```
function approveRecommendation(recommendation, setState) {
  setState(prev => ({
    ...prev,
    buses: prev.buses.map(bus =>
      bus.id === recommendation.busId
        ? {...bus, route:recommendation.routeId,
           status:"Deployed"}
        : bus
    ),
    alerts: prev.alerts.filter(
      alert => alert.routeId !== recommendation.routeId
    )
  }));
}
```

20. Simulated Real-Time Demo

```
setInterval(() => {
  setBuses(prev => prev.map(bus => ({
    ...bus,
    occupancy: Math.max(
      15, Math.min(100,
      bus.occupancy + Math.round((Math.random()-0.5)*6))
    ),
    eta: Math.max(1, bus.eta + (Math.random() > 0.5 ? 1 : -1))
  })));
}, 5000);
```

In the final implementation, cleanup the interval inside a React useEffect return function to avoid memory leaks when the page unmounts.

21. Demo Scenario

```
08:15 AM
Route 12 current occupancy = 82%
        ↓
Click "Predict Demand"
        ↓
Predicted occupancy = 97%
Risk = HIGH
Window = 15 minutes
        ↓
AI recommendation:
Deploy Bus 24 to Route 12
        ↓
Admin clicks Approve
        ↓
Bus 24 status = Deployed
Route 12 alert is resolved
```

22. Test Cases

ID	Test	Expected Result
TC01	Open dashboard	Stats and charts load
TC02	Select Route 12	Route selected
TC03	Predict demand	Prediction appears
TC04	High-risk result	Alert appears
TC05	Approve Bus 24	Bus state updates
TC06	Open map	Markers render
TC07	Passenger view	Buses displayed
TC08	Mobile layout	No horizontal overflow
TC09	API unavailable	Mock data works
TC10	Invalid route	Validation message

23. Advanced ML Upgrade

For a production-oriented version, replace the transparent weighted prototype function with a trained regression or time-series model. Candidate features include hour, weekday, holiday, historical demand, current occupancy, traffic index, weather index and special-event indicators.

```
import pandas as pd
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split

df = pd.read_csv("passenger_demand.csv")

features = [
    "hour","weekday","holiday","traffic_index",
    "weather_index","current_occupancy",
    "historical_average"
]

X = df[features]
y = df["future_passengers"]

X_train,X_test,y_train,y_test = train_test_split(
    X,y,test_size=0.20,random_state=42
)

model = RandomForestRegressor(
    n_estimators=200, random_state=42
)
model.fit(X_train,y_train)
prediction = model.predict(X_test)
```

24. Evaluation Metrics

• MAE for average prediction error.

• RMSE for larger-error sensitivity.

• Precision/Recall for overcrowding-risk classification.

• MAPE only when zero-demand cases are handled correctly.

• Operational metrics such as waiting-time reduction and bus utilization.

25. Security & Privacy

• Use authentication and role-based admin access.

• Keep API keys and database credentials in environment variables.

• Use HTTPS in deployment.

• Minimize personally identifiable passenger data.

• Aggregate or anonymize passenger counts where possible.

• Camera-based counting should be optional and subject to privacy governance.

26. Limitations

• Prototype prediction values are simulated and are not a validated production ML model.

• Real GPS, ticketing and traffic integrations require reliable external data.

• Optimization must include operational constraints before real deployment.

• Camera counting requires privacy controls and accuracy testing.

• Real-world deployment needs model validation, security review and transport-authority approval.

27. Future Scope

• Dynamic route changes during events.

• Predictive bus maintenance.

• Personalized less-crowded route recommendations.

• Electric bus charging and energy optimization.

• Edge AI for real-time passenger counting.

• Integration with smart-city transport control centers.

28. Expected Results

• The prototype predicts near-future occupancy for selected routes.

• High-risk routes generate an overcrowding warning.

• The optimization service ranks routes by operational priority.

• Admin approval changes the simulated fleet state.

• Passenger view can recommend a lower-crowding alternative.

29. Conclusion

TransitAI demonstrates how AI can move public transportation from reactive management toward predictive decision support. The prototype connects demand prediction, overcrowding alerts, live transport visibility and route optimization in one system.

Core message: DETECT → PREDICT → OPTIMIZE → ACT.

30. References / Learning Resources

• React documentation – component-based frontend development.

• Node.js and Express documentation – backend API development.

• Python and scikit-learn documentation – machine learning and evaluation.

• Leaflet documentation – interactive maps.

• Recharts documentation – data visualization.

Academic note: the code in this document is a prototype reference. The final submission should include the actual source files produced and tested in the working project.